

# LAPORAN PENUGASAN KELOMPOK

## Solving Problems by Searching

*Tema: Pathfinding NPC pada Permainan Dungeon-like*



### By Kelompok 3

Anggota Kelompok :

- |                              |            |
|------------------------------|------------|
| 1. Diva Rajestiadi           | (11221015) |
| 2. Muhammad Haikal Ar Ridwan | (11221088) |
| 3. Muhammad Adzikril         | (11251073) |
| 4. Pradifta Alfariz          | (11251035) |
| 5. Vivian Marsyanda          | (11231091) |

## 1. Deskripsi Permasalahan

Dalam suatu permainan, Non-Player Character (NPC) bergerak secara otomatis dari satu titik ke titik yang lain sesuai dengan tujuan yang dimilikinya -- misalnya NPC musuh yang mengejar pemain, atau NPC yang menuntun pemain menuju lokasi tujuan. Pergerakan NPC ini diproses melalui pencarian rute (pathfinding) pada representasi peta/area yang telah dipetakan.

Pada tugas ini, permasalahan yang diangkat adalah: NPC berada pada titik Spawn di dalam sebuah peta area dungeon dan harus menemukan rute menuju Boss Area, melalui berbagai ruang perantara/jalan yang akan dilalui oleh player.

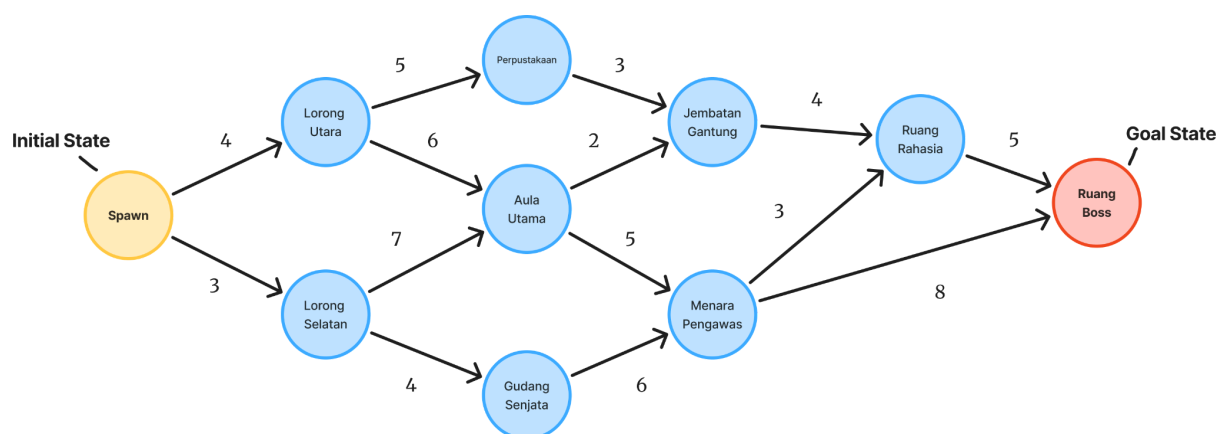
## 2. Formulasi Search Problem

| Komponen         | Deskripsi                                                                     |
|------------------|-------------------------------------------------------------------------------|
| Initial State    | NPC berada di node Spawn (titik awal permainan).                              |
| Actions          | Bergerak dari area (node) ke area yang lain lewat jalan (edge) yang tersedia. |
| Transition Model | NPC kembali mencari cabang jalan (edge) setelah berada di area (node) baru.   |
| Goal State       | NPC berhasil menuju ruangan boss.                                             |
| Path Cost        | Jumlah cost/jarak yang ditempuh oleh NPC sepanjang area.                      |

Mengapa masalah ini dapat diselesaikan dengan pendekatan search?

Peta dungeon dapat direpresentasikan sebagai graph diskrit dengan sejumlah state (ruang) yang berhingga, actions yang jelas dan terbatas (hanya bisa berpindah ke ruang yang terhubung langsung), serta goal state yang spesifik yaitu mencapai area boss. Karena ruang keadaan bersifat diskrit dan dapat diperhitungkan, permasalahan ini dapat dimodelkan sebagai search problem tanpa memerlukan pendekatan rumit.

## 3. State Space



Gambar 1 Visualisasi Graph Node Peta Dungeon

### 3.1 Daftar Node (10 node)

| Node  | Nama Area                       | Koordinat (x, y) |
|-------|---------------------------------|------------------|
| Spawn | Titik masuk NPC (Initial State) | 0, 0             |

| Node             | Nama Area                          | Koordinat (x, y) |
|------------------|------------------------------------|------------------|
| Lorong utara     | Lorong sisi utara                  | 1, 2             |
| Lorong selatan   | Lorong sisi selatan                | 1, -2            |
| Perpustakaan     | Ruang perpustakaan kuno            | 3, 3             |
| Aula utama       | Aula tengah dungeon                | 3, 1             |
| Gudang senjata   | Gudang penyimpanan senjata         | 3, -3            |
| Jembatan gantung | Jembatan penghubung                | 5, 2             |
| Menara pengawas  | Menara pengawas musuh              | 5, -1            |
| Ruang rahasia    | Ruang tersembunyi                  | 7, 1             |
| Ruang bos        | Ruang pertarungan bos (Goal State) | 9, 0             |

Koordinat digunakan sebagai dasar perhitungan heuristic (Euclidean/Manhattan distance) untuk  $A^*$ .

### 3.2 Daftar Edge & Cost (14 edge)

| Dari             | Ke               | Cost |
|------------------|------------------|------|
| Spawn            | Lorong Utara     | 4    |
| Spawn            | Lorong Selatan   | 3    |
| Lorong Utara     | Perpustakaan     | 5    |
| Lorong Utara     | Aula Utama       | 6    |
| Lorong Selatan   | Gudang Senjata   | 4    |
| Lorong Selatan   | Aula Utama       | 7    |
| Perpustakaan     | Jembatan Gantung | 3    |
| Aula Utama       | Jembatan Gantung | 2    |
| Aula Utama       | Menara Pengawas  | 5    |
| Gudang Senjata   | Menara Pengawas  | 6    |
| Jembatan Gantung | Ruang Rahasia    | 4    |
| Menara Pengawas  | Ruang Rahasia    | 3    |
| Ruang Rahasia    | Ruang Bos        | 5    |
| Menara Pengawas  | Ruang Bos        | 8    |

Graph bersifat tak berarah (undirected) — NPC bisa bergerak bolak-balik antar ruang yang terhubung.

### 3.3 Minimal 3 Alternatif Jalur dari Spawn ke RuangBos

1. Spawn > LorongUtara > Perpustakaan >JembatanGantung > RuangRahasia > RuangBos  
Node = 6 || Cost = 4+5+3+4+5 = 21
2. Spawn > Lorong Utara > AulaUtama > JembatanGantung > RuangRahasia > RuangBos  
Node = 6 || Cost = 4+6+2+4+5 = 21
3. Spawn > LorongSelatan > GudangSenjata > MenaraPengawas > RuangBos  
Node = 5 || Cost = 3+4+6+8 = 21
4. Spawn > LorongUtara > AulaUtama > MenaraPengawas > RuangBos  
Node = 5 || Cost = 4+6+5+8 = 23

## 4. Implementasi BFS

```
Initial State : Spawn
Goal State    : RuangBos
Expansion Order : Spawn -> LorongSelatan -> LorongUtara -> AulaUtama ->
GudangSenjata -> Perpustakaan -> JembatanGantung -> MenaraPengawas -> RuangRahasia -
> RuangBos
Solution Path  : Spawn -> LorongSelatan -> AulaUtama -> MenaraPengawas -> RuangBos
Number of Steps : 4
Path Cost      : 23
Expanded Nodes  : 10
```

BFS diimplementasikan menggunakan struktur data *deque* (queue FIFO) dari modul *collections*. Node dimasukkan ke frontier dan diambil selalu dari depan (*popleft()*), sehingga node dengan kedalaman lebih dangkal selalu diekspansi lebih dulu. Setiap node yang sudah dikunjungi ditandai di *visited* agar tidak diekspansi ulang. Ketika node yang diambil sama dengan goal, jalur ditelusuri balik melalui dictionary *parent*.

## 5. Implementasi DFS

```
Solution Path : Spawn -> LorongSelatan -> AulaUtama -> JembatanGantung ->
RuangRahasia -> RuangBos
Number of Steps : 5
Path Cost       : 21
Expanded Nodes  : 10
```

DFS diimplementasikan menggunakan *list* Python sebagai stack (LIFO) — node diambil dengan *pop()* dari ujung list. Tetangga diurutkan secara alfabetis terbalik sebelum dimasukkan ke stack, supaya urutan penjelajahan tetap deterministik (alfabetis) saat di-pop. Node ditandai *visited* saat diambil dari stack, bukan saat dimasukkan, untuk menghindari duplikasi kunjungan.

## 6. Implementasi UCS

```
Solution Path : Spawn -> LorongSelatan -> GudangSenjata -> MenaraPengawas ->
RuangBos
Number of Steps : 4
Path Cost       : 21 (OPTIMAL)
Expanded Nodes  : 10
```

UCS menggunakan *heapq* sebagai priority queue, di mana prioritas ditentukan oleh *cost\_so\_far* ( $g(n)$ ) — bukan urutan masuk. Setiap kali ditemukan jalur baru menuju suatu node dengan cost lebih kecil dari yang tercatat sebelumnya, *cost\_so\_far* dan *came\_from* diperbarui (*relaxation*), lalu node dimasukkan lagi ke heap. Tie-breaker berupa counter ditambahkan pada tuple (*cost*, *counter*, *node*) agar *heapq* tidak membandingkan string saat cost sama.

## 7. Implementasi A\*

```
Solution Path : Spawn -> LorongSelatan -> GudangSenjata -> MenaraPengawas -> RuangBos
Number of Steps : 4
Path Cost      : 21 (OPTIMAL)
Expanded Nodes : 10
```

| Node           | g(n)  | h(n) | f(n)  |
|----------------|-------|------|-------|
| Spawn          | 0.00  | 9.00 | 9.00  |
| LorongSelatan  | 3.00  | 8.25 | 11.25 |
| GudangSenjata  | 7.00  | 6.71 | 13.71 |
| MenaraPengawas | 13.00 | 4.12 | 17.12 |
| RuangBos       | 21.00 | 0.00 | 21.00 |

A\* mirip UCS, namun prioritas heap ditentukan oleh  $f(n) = g(n) + h(n)$ , di mana  $h(n)$  dihitung lewat fungsi `heuristic()` (Euclidean distance ke goal) dari `heuristic.py`. Nilai g, h, f tiap node disimpan di dictionary `node_values` untuk keperluan pelaporan.

## 8. Eksperimen

### Eksperimen 1 — Kondisi Normal

| Algoritma | Path Cost | Steps | Expanded Nodes | Optimal?       |
|-----------|-----------|-------|----------------|----------------|
| BFS       | 23        | 4     | 10             | Tidak          |
| DFS       | 21        | 5     | 10             | Ya (kebetulan) |
| UCS       | 21        | 4     | 10             | Ya             |
| A*        | 21        | 4     | 10             | Ya             |

### Eksperimen 2 — Perubahan Path Cost

Perubahan yang dilakukan:

- **AulaUtama–JembatanGantung**  $2 \rightarrow 9$
- **LorongSelatan–GudangSenjata**  $4 \rightarrow 1$
- **MenaraPengawas–RuangBos**  $8 \rightarrow 2$ .

| Algoritma | Path Cost (baru)     | Steps | Expanded Nodes    | Optimal? |
|-----------|----------------------|-------|-------------------|----------|
| BFS       | 17 (berubah dari 23) | 4     | 10                | Tidak    |
| DFS       | 28 (berubah dari 21) | 5     | 10                | Tidak    |
| UCS       | 12 (berubah dari 21) | 4     | 9                 | Ya       |
| A*        | 12 (berubah dari 21) | 4     | 6 (turun dari 10) | Ya       |

Perubahan cost paling memengaruhi **UCS dan A\*** karena keduanya menentukan urutan ekspansi berdasarkan cost aktual ( $g(n)$ ); jalur yang tadinya mahal (lewat GudangSenjata) menjadi jalur termurah setelah cost-nya diturunkan. BFS tidak terpengaruh cara memilih jalurnya (tetap steps tersedikit) tapi *nilai* cost-nya berubah karena cost tiap edge berubah. DFS paling tidak terprediksi — solusinya berubah drastis (21→28) karena urutan eksplorasi DFS sensitif terhadap urutan tetangga, bukan terhadap cost.

Menariknya, *A menjadi jauh lebih efisien\** (6 node vs 9 node UCS) karena heuristic mengarahkannya langsung ke jalur **LorongSelatan → GudangSenjata → MenaraPengawas → RuangBos** tanpa perlu mengeksplorasi cabang lain.

### Eksperimen 3 — Perubahan Heuristic

Bandingkan A\* dengan 4 heuristic (semua pada graph asli):

| Heuristic A*                    | Path Cost | Expanded Nodes | Optimal?       |
|---------------------------------|-----------|----------------|----------------|
| Euclidean (default)             | 21        | 10             | Ya             |
| Manhattan                       | 21        | 10             | Ya             |
| NoI (h=0, setara UCS)           | 21        | 10             | Ya             |
| Inflated (×3, tidak admissible) | 21        | 5              | Ya (kebetulan) |

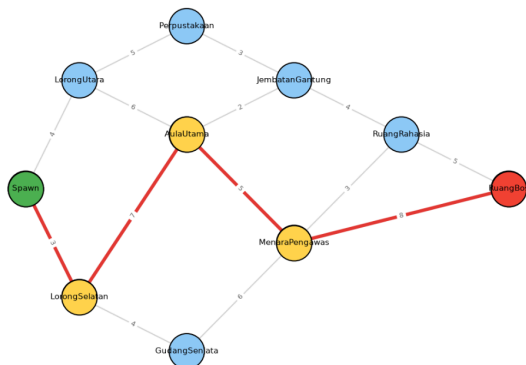
Ketiga heuristic pertama (Euclidean, Manhattan, nol) menghasilkan jumlah ekspansi node yang **sama** (10) karena skala heuristic-nya relatif kecil dibanding cost edge, sehingga tidak cukup kuat mengarahkan pencarian lebih awal.

Heuristic **inflated (×3)** justru mengurangi ekspansi node secara drastis (10 → 5) karena nilai  $h$  yang besar membuat A\* sangat "yakin" ke arah goal dan langsung fokus ke satu jalur. Namun ini **kebetulan tetap optimal** — pada graph lain, heuristic yang tidak *admissible* seperti ini berisiko membuat A melewati solusi optimal\*, karena bisa mengecualikan jalur yang sebenarnya lebih murah hanya karena estimasinya "kelihatan" lebih jauh.

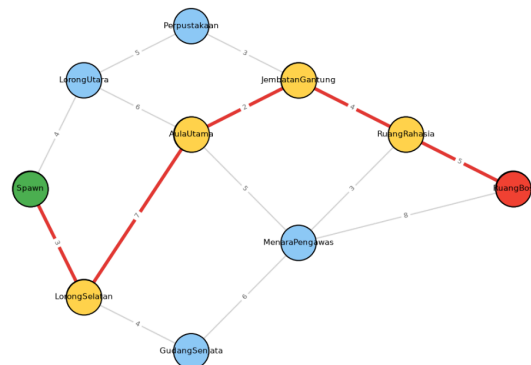
## 9. Perbandingan Algoritma

**Perbandingan Solution Path: BFS vs DFS vs UCS vs A\***

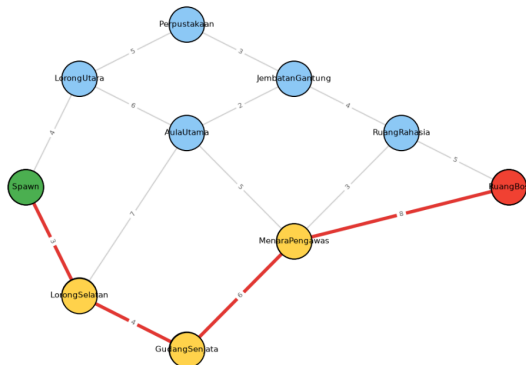
**BFS** (cost=23, steps=4, expanded=10)



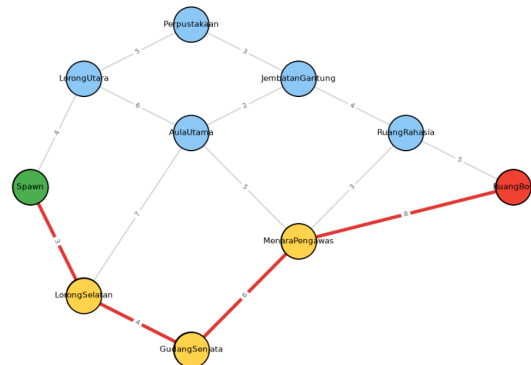
**DFS** (cost=21, steps=5, expanded=10)



**UCS** (cost=21, steps=4, expanded=10)



**A\*** (cost=21, steps=4, expanded=10)



| Algoritma | Solution Path                                                                 | Path Cost | Steps | Expanded Nodes | Apakah Optimal?   |
|-----------|-------------------------------------------------------------------------------|-----------|-------|----------------|-------------------|
| BFS       | Spawn → LorongSelatan → AulaUtama → MenaraPengawas → RuangBos                 | 23        | 4     | 10             | Tidak             |
| DFS       | Spawn → LorongSelatan → AulaUtama → JembatanGantung → RuangRahasia → RuangBos | 21        | 5     | 10             | Ya<br>(kebetulan) |
| UCS       | Spawn → LorongSelatan → GudangSenjata → MenaraPengawas → RuangBos             | 21        | 4     | 10             | Ya                |
| A*        | Spawn → LorongSelatan → GudangSenjata → MenaraPengawas → RuangBos             | 21        | 4     | 10             | Ya                |

## 10. Analisis dan Refleksi

### 1. Apakah BFS menghasilkan path cost minimum pada graph Anda?

Tidak. Pada Eksperimen 1, BFS menghasilkan jalur Spawn → LorongSelatan → AulaUtama → MenaraPengawas → RuangBos dengan path cost 23, sedangkan cost minimum pada graph ini adalah 21 (dicapai oleh DFS, UCS, dan A\*). Hal ini terjadi karena BFS hanya mengutamakan jumlah langkah (steps) paling sedikit, dalam kasus ini 4 langkah, tanpa mempertimbangkan besar cost pada masing-masing edge. Akibatnya, jalur dengan langkah tersedikit belum tentu jalur dengan total cost terendah. Tabel perbandingan algoritma juga menandai hasil BFS ini dengan "Tidak" pada kolom optimalitas.

### 2. Apakah DFS menghasilkan solusi optimal pada graph Anda?

Ya, DFS memperoleh jalur Spawn → LorongSelatan → AulaUtama → JembatanGantung → RuangRahasia → RuangBos dengan path cost 21, sama dengan cost minimum. Namun hal ini bersifat kebetulan, sesuai keterangan pada tabel yang menuliskan "Ya". DFS tidak mempertimbangkan cost sama sekali dalam menentukan urutan ekspansi node; ia hanya mengikuti urutan tetangga yang diurutkan alfabetis terbalik lalu diambil dari stack. Hal ini dibuktikan pada Eksperimen 2, di mana setelah cost beberapa edge diubah, hasil DFS berubah drastis dari 21 menjadi 28, sementara BFS dan UCS tetap konsisten dengan logikanya masing-masing. Ini menunjukkan bahwa DFS sensitif terhadap urutan eksplorasi, bukan terhadap nilai cost, sehingga optimalitasnya pada graph ini bukan jaminan struktural, melainkan kebetulan.

### 3. Mengapa hasil BFS dan DFS dapat berbeda?

BFS: Mengunjungi tetangga terdekat pada level yang sama terlebih dahulu sebelum turun ke level berikutnya

DFS: Menyelami satu jalur atau cabang sejauh mungkin sampai ujung, baru kemudian menelusuri balik ke cabang lain.

### 4. Mengapa UCS dapat memilih jalur berbeda dari BFS ketika edge memiliki cost berbeda?

UCS dapat memilih jalur berbeda dari BFS ketika edge memiliki cost berbeda karena keduanya menggunakan kriteria pemilihan node yang berbeda:

UCS memilih node berdasarkan total cost terkecil dari titik awal. sedangkan BFS memilih node berdasarkan jumlah edge.

Karena itu, ketika cost edge berbeda, jalur terpendek secara jumlah edge belum tentu jalur termurah, sehingga hasil BFS dan USC berbeda.

### 5. Bagaimana A\* menentukan node yang akan diekspansi?

A\* selalu mengekspansi node dengan nilai  $f(n) = g(n) + h(n)$  terkecil di frontier (priority queue) – kombinasi antara biaya aktual yang sudah ditempuh dari Start ( $g(n)$ ) dan estimasi biaya menuju Goal ( $h(n)$ ), dihitung dari heuristic Euclidean berdasarkan koordinat).

### 6. Apa pengaruh nilai heuristic terhadap proses pencarian?

Heuristic menentukan seberapa “terarah” pencarian A\*. Pada Eksperimen 3, heuristic dengan skala kecil (Euclidean/Manhattan) menghasilkan jumlah ekspansi yang sama (10 node) karena belum cukup kuat mengarahkan pencarian. Namun heuristic *inflated* (x3) memangkas

ekspansi drastis menjadi hanya 5 node, karena nilai  $h$  yang besar membuat  $A^*$  langsung “yakin” ke satu arah dan tidak banyak mengeksplorasi cabang lain. Semakin besar dan akurat heuristic (tanpa melebih-lebihkan), semakin efisien pencarian  $A^*$ .

### 7. Apa yang terjadi ketika path cost diubah?

Jika path cost berubah, algoritma UCS dan  $A^*$  dapat menghasilkan solution path yang berbeda karena keduanya mempertimbangkan biaya perjalanan dalam menentukan jalur. BFS dan DFS tidak secara langsung mempertimbangkan path cost, sehingga jalur yang dipilih cenderung tetap selama struktur graph dan urutan node tidak berubah. Namun, nilai path cost dan status optimalitas dari BFS atau DFS dapat berubah.

### 8. Apa yang terjadi ketika heuristic diubah?

Perubahan heuristic pada  $A^*$  dapat memengaruhi urutan node dan jalur yang dipilih. Heuristic yang lebih informatif dapat membuat  $A^*$  lebih efisien dengan mengurangi jumlah node yang diekspansi.

### 9. Algoritma mana yang menghasilkan path cost terendah?

Algoritma yang menghasilkan path cost terendah adalah DFS, UCS, dan  $A^*$ , yaitu sebesar 21. Namun, UCS dan  $A^*$  lebih dapat diandalkan karena keduanya memang mempertimbangkan nilai cost dalam pencarian jalur. DFS memperoleh cost 21 hanya secara kebetulan akibat urutan eksplorasinya.

### 10. Algoritma mana yang melakukan ekspansi node paling sedikit?

Pada kondisi normal, semua algoritma (BFS, DFS, UCS, dan  $A^*$ ) melakukan jumlah ekspansi node yang sama, yaitu 10 node. Jadi, tidak ada satu algoritma yang paling sedikit melakukan ekspansi pada eksperimen ini. Namun, pada eksperimen perubahan heuristic,  $A^*$  dengan heuristic inflated ( $\times 3$ ) menjadi yang paling sedikit melakukan ekspansi, yaitu hanya 5 node.

### 11. Algoritma mana yang paling sesuai untuk kasus NPC pathfinding ini, dan mengapa?

$A^*$  adalah pilihan yang paling sesuai untuk kasus NPC pathfinding. Dikarenakan agent telah mengetahui posisi Goal state dan langsung mempertimbangkan jalur yang diambil. Efisiensi ini penting karena pathfinding akan dihitung secara real-time dan berulang kali, yang akan berpengaruh kedalam optimalisasi permainan.

## 11. Kesimpulan

Berdasarkan seluruh eksperimen yang telah dilakukan pada kasus **NPC Pathfinding**, dapat disimpulkan bahwa keempat algoritma pencarian memiliki karakteristik dan tingkat keandalan yang berbeda dalam menyelesaikan permasalahan yang sama.

$A^*$  merupakan algoritma yang **paling sesuai untuk kasus NPC pathfinding** dalam permainan, karena menggabungkan dua keunggulan sekaligus: **menjamin path cost optimal** seperti UCS, namun dengan **efisiensi komputasi yang jauh lebih baik** berkat pemanfaatan informasi heuristic menuju goal. Karakteristik ini sangat relevan bagi kebutuhan NPC dalam game, yang harus menghitung rute secara *real-time* dan berulang kali, sehingga keseimbangan antara akurasi solusi dan kecepatan komputasi menjadi faktor penentu utama dalam pemilihan algoritma pencarian.